

Network-Segmentation

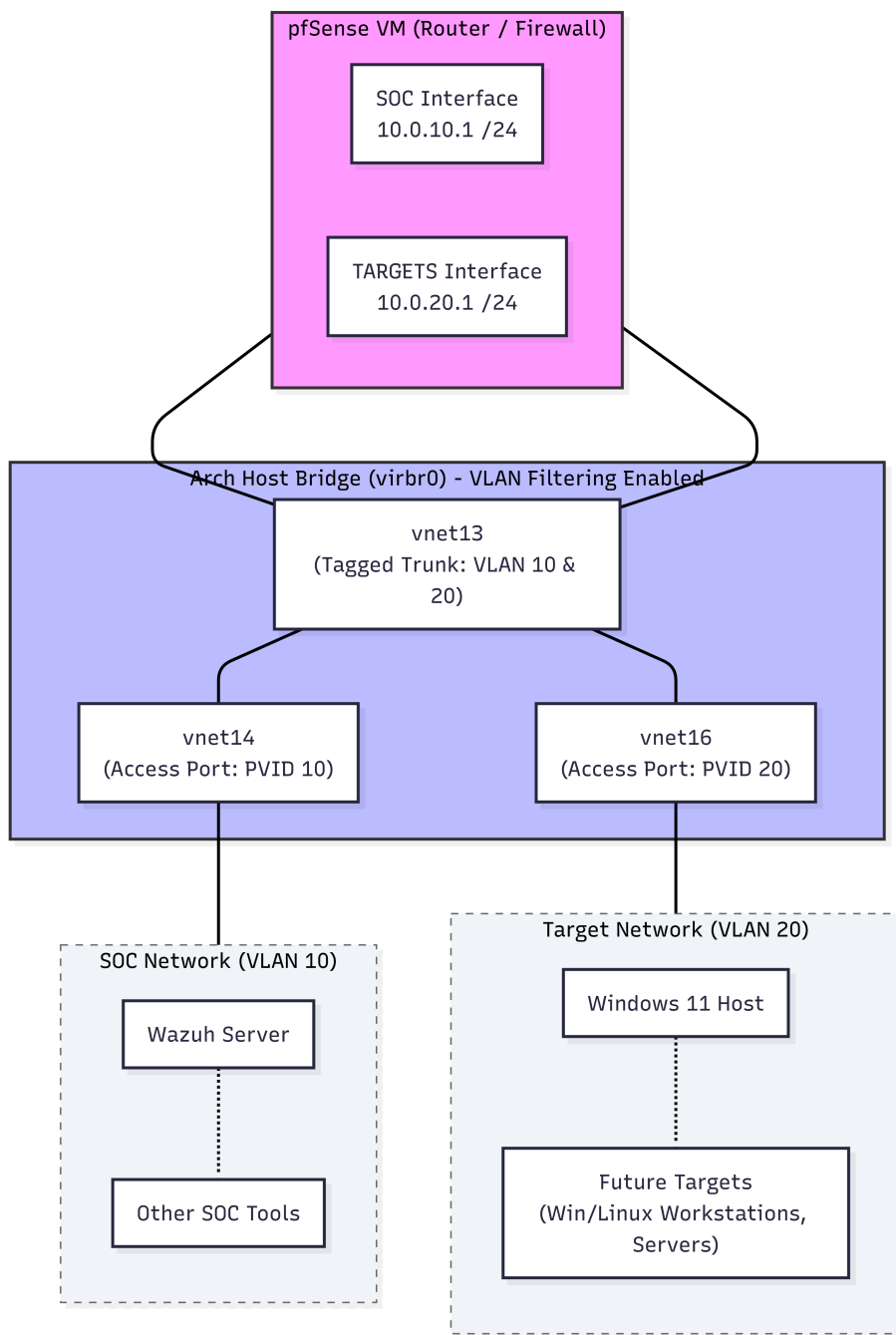
BELASRI Ayman

1 Introduction

To secure our lab environment, we segmented the virtual network into separate subnets using VLANs managed by pfSense. This separates our Security Operations Center (SOC) infrastructure from our monitored target machines.

2 Setup Overview

To achieve full network segmentation while maintaining controlled access, we enabled VLAN filtering on the host bridge (`virbr0`) to isolate traffic, linked the subnets to pfSense through a single 802.1Q trunk port, mapped each VM to its dedicated untagged access port, and configured pfSense firewall rules to restrict inter-VLAN routing between the SOC and target networks.



3 VLAN Creation & Interface Setup

We defined the subnets and virtual interfaces directly within the pfSense WebGUI.

- **Created 802.1Q VLAN Interfaces:**
 - **VLAN 10:** Assigned to parent interface `vtnet1` (Name: SOC, Subnet: 10.0.10.0/24).
 - **VLAN 20:** Assigned to parent interface `vtnet1` (Name: TARGETS, Subnet: 10.0.20.0/24).

Interfaces / VLANs

Interface Assignments Interface Groups Wireless **VLANs** QinQs PPPs GREs GIFs Bridges LAGGs

VLAN Interfaces				
Interface	VLAN tag	Priority	Description	Actions
vtnet1 (lan)	10		SOC_VLAN	
vtnet1 (lan)	20		TARGET_VLAN	

+ Add

Interface	Network port
WAN	vtnet0 (52:54:00:5f:23:07)
LAN	vtnet1 (52:54:00:47:8d:7e)
SOC	VLAN 10 on vtnet1 - lan (SOC_VLAN)
TARGET	VLAN 20 on vtnet1 - lan (TARGET_VLAN)

Save

- **Configured Gateways & Services:**
 - Gateway IPs set: 10.0.10.1 (VLAN 10) and 10.0.20.1 (VLAN 20).
 - Enabled DHCP server pools for both interfaces (e.g., 10.0.20.100–10.0.20.200 on VLAN 20).

General DHCP Options	
DHCP Backend	ISC DHCP
Enable	☒ Enable DHCP server on TARGET interface
BOOTP	☐ Ignore BOOTP queries
Deny Unknown Clients	Allow all clients When set to Allow all clients, any DHCP client will get an IP address within this scope/range on this interface. If set to Allow known clients from any interface, any DHCP client with a MAC address listed in a static mapping on any scope(s)/interface(s) will get an IP address. If set to Allow known clients from only this interface, only MAC addresses listed in static mappings on this interface will get an IP address within this scope/range.
Ignore Denied Clients	☐ Ignore denied clients rather than reject This option is not compatible with failover and cannot be enabled when a Failover Peer IP address is configured.
Ignore Client Identifiers	☐ Do not record a unique identifier (UID) in client lease data if present in the client DHCP request This option may be useful when a client can dual boot using different client identifiers but the same hardware (MAC) address. Note that the resulting server behavior violates the official DHCP specification.
Primary Address Pool	
Subnet	10.0.20.0/24
Subnet Range	10.0.20.1 - 10.0.20.254
Address Pool Range	10.0.20.100 10.0.20.200 From To The specified range for this pool must not be within the range configured on any other address pool for this interface.
Additional Pools	+ Add Address Pool If additional pools of addresses are needed inside of this subnet outside the above range, they may be specified here.
Server Options	

- **Configured Firewall Rules:**
 - Added rules under **Firewall > Rules** for both interfaces, permitting required traffic (e.g., allowing ICMP/IPv4 out).

4 Wazuh VM Networking Cleanup

To prevent routing loops and ARP failures, duplicate IP assignments on the Wazuh guest OS were removed.

- **Separated Interface Duties:**
 - **eth0**: Kept strictly for management/NAT.
 - **eth1**: Designated for SOC network traffic (10.0.10.5/24).
- **Cleared IP & Routing Conflicts:**

```
sudo ip addr del 10.0.10.5/24 dev eth0
sudo ip addr del 192.168.1.50/24 dev eth1
sudo ip route replace default via 10.0.10.1 dev eth1
```

5 Host Bridge VLAN Filtering

The libvirt software bridge (virbr0) was switched from a flat Layer 2 switch to a VLAN-aware bridge to support 802.1Q tagging.

- **Enabled VLAN Filtering on Bridge:**

```
sudo ip link set virbr0 type bridge vlan_filtering 1
```

6. XML Modifications for Static TAP Devices

In Virt-Manager , we add `<target dev='...'/>` inside `<interface type='network'>` to fix interface names across reboots:

- **pfSense (pfsense)**: Add `<target dev='tap-pfsense'/>` to 2nd NIC (Trunk/LAN).
- **Windows 11 (win11)**: Add `<target dev='tap-win11'/>` to the NIC.
- **Wazuh SOC (wazuh)**: Add `<target dev='tap-wazuh'/>` to 2nd NIC (eth1).

7. Bridge Port Assignments

To wrap up our network setup, we configured our bridge ports using `bridge vlan` to map out our tagged trunks and untagged access ports. By switching to static TAP interface names (`tap-pfsense`, `tap-wazuh`, and `tap-win11`), we ensured our VLAN configuration stays intact across reboots without running into libvirt hook deadlocks.

7.1 Tagged Trunk Port (pfSense)

Since pfSense needs raw 802.1Q tagged frames passed directly to its `vtnet1` interface, we set up `tap-pfsense` as a trunk port for both VLANs:

```
sudo bridge vlan add dev tap-pfsense vid 10 tagged
sudo bridge vlan add dev tap-pfsense vid 20 tagged
```

7.2 Access Port: SOC

Our Wazuh instance sends standard untagged traffic, so we designated `tap-wazuh` as an access port for VLAN 10. The host bridge tags any incoming frames with VLAN 10 and strips the tag when sending traffic back out to the VM:

```
sudo bridge vlan add dev tap-wazuh vid 10 pvid untagged
sudo bridge vlan del dev tap-wazuh vid 1 2>/dev/null
```

7.3 Access Port: Target Hosts

Similarly, Windows 11 handles untagged traffic, so we assigned `tap-win11` to access VLAN 20. The bridge tags incoming packets with VLAN 20 on the way in and untags them on the way out:

```
sudo bridge vlan add dev tap-win11 vid 20 pvid untagged
sudo bridge vlan del dev tap-win11 vid 1 2>/dev/null
```